StudyRoute: technical documentation

International Track, Topic C: Cross-border and Student Finance Assistant. Covers the three items the rules ask for: **system architecture**, **core algorithm description** and **security design**.

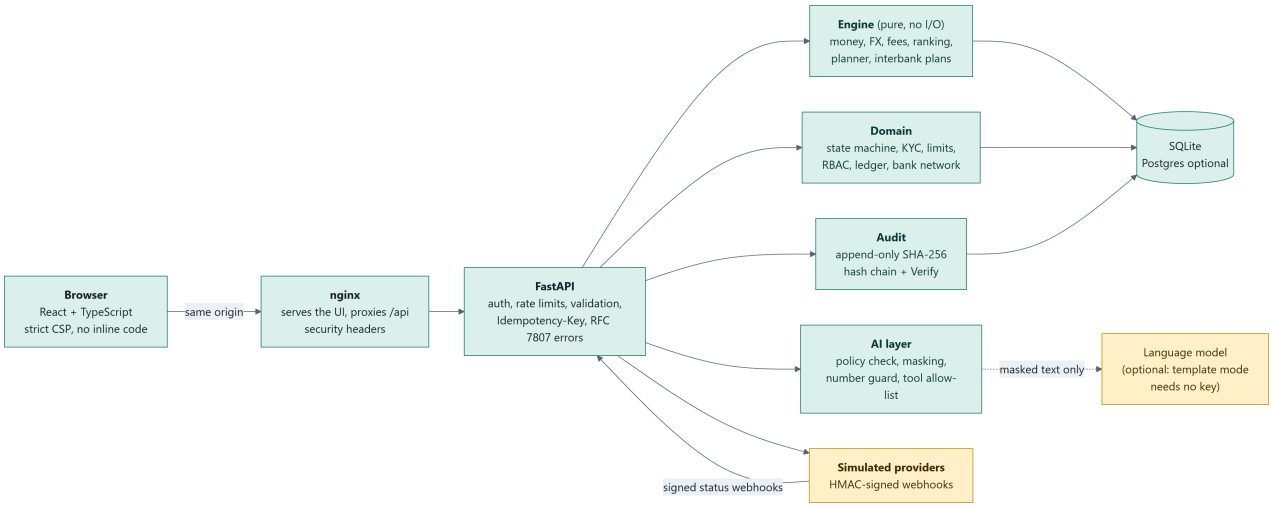
> StudyRoute is a sandbox. No real money moves and all data is synthetic. Guidance is indicative: confirm with your Authorised Dealer (AD) bank.

Design principle. The engine calculates, the AI explains, the human approves, and every step is logged. Each part below exists to make one of those four statements true and testable.

---

1. System architecture

1.1 Components



```
 Browser (Next.js static export, React + TypeScript)
        |  same origin, strict CSP
        v
     nginx  --- serves the built UI, proxies /api, sets X-Forwarded-For
        |
        v
 FastAPI  (security middleware: request id, size limit, headers, X-Sandbox)
   |   auth (JWT ES256) - RBAC - rate limit - schema validation - Idempotency-Key - RFC 7807
errors
   |
   +--> Engine    pure functions: money, FX margin, fees, ranking, planner, runway, MT103
preview
   +--> Domain    state machine, KYC gate, limits, sanctions check, recipients, documents, rules
   +--> AI layer  policy check, facts, masking, model client (optional), number guard, tool
guard
   +--> Audit     append-only hash chain, verify
   +--> Providers mock adapters A, B, C  -->  simulator  -->  HMAC-signed webhooks back to the
API
   +--> Bank network  invented banks, clearing houses and a correspondent  -->  double-entry
ledger
   +--> Sandbox chain Solidity contracts on an in-memory EVM: escrow, tokenised deposit, audit
anchor
   |
   v
 SQLite (Postgres optional), foreign keys on, CHECK constraints, audit triggers
```

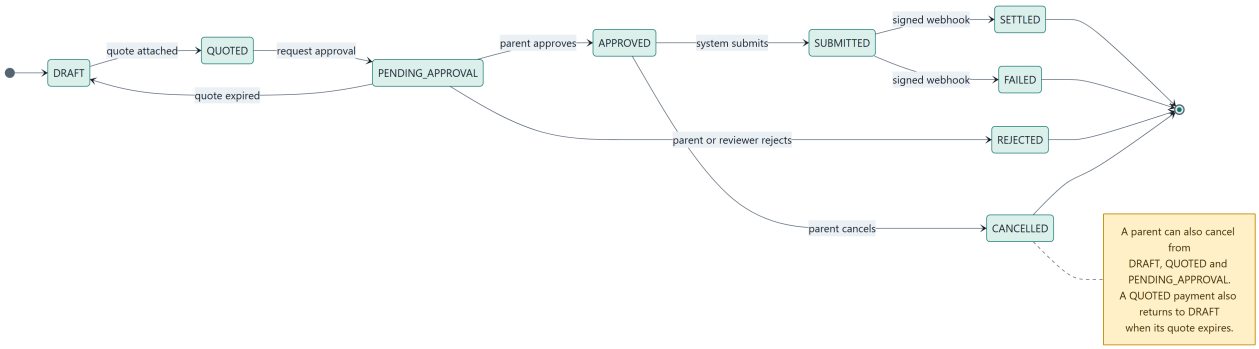
Layer	Folder	Responsibility	I/O
Engine	`backend/app/engine/`	Every number: costs, ranking, deadline, runway, the student budget (`student_budget.py`), and the journals of a bank transfer (`interbank.py`)	None (pure)
Domain	`backend/app/domain/`	State machine, guards, RBAC, KYC, limits, config, the ledger, the bank network and the student guide (`student_guide.py`)	Database via a passed-in session
API	`backend/app/api/`	Routes, authentication, idempotency, rate limits, errors	HTTP
Audit	`backend/app/audit/`	Hash chain and verification	Database
Chain	`backend/app/chain/` , `contracts/`	Compiled contracts, ABI codec and the in-memory EVM (`py-evm`)	None (memory)
Providers	`backend/app/providers/`	Mock providers, simulator, webhook signing	HTTP (webhooks)
AI	`backend/app/ai/`	Orchestrator, masking, number guard, tools, templates	Optional network call to a model
Web	`web/`	Dashboard, compare, payments, recipients, documents, student guide, audit, rules, bank network, smart escrow, sandbox, assistant	HTTP

The engine has no database, clock or network access, so it is fully unit-testable and its output can be trusted as the single source of numbers.

1.2 Data model

`users`, `recipients` (account stored encrypted with AES-256-GCM, only the last four digits shown), `quotes` (inputs, results, mid-rate source and time, input hash), `payments` (state, summary hash, version for optimistic locking), `payment_transitions`, `audit_events`, `audit_head`, `idempotency_keys` (unique per user and key), `webhook_events` (unique event id), `config_rules` (value, source URL, last checked, verified), `kyc_records`, `documents`, `mfa_proofs`, `refresh_tokens`, `provider_ledger`, and for the bank network `bank_accounts`, `bank_transfers`, `bank_transfer_events`, `ledger_journals` and `ledger_entries`. The payment state is limited by a CHECK constraint, and triggers block `UPDATE` and `DELETE` on `audit_events`, `ledger_journals` and `ledger_entries`.

1.3 Payment lifecycle




```
DRAFT -> QUOTED -> PENDING_APPROVAL -> APPROVED -> SUBMITTED -> SETTLED
            |            |                |             |-----> FAILED
            |            |-> REJECTED     |-> CANCELLED
            +--- quote expires: QUOTED or PENDING_APPROVAL returns to DRAFT
```

It is a transition table: every row names the event, the allowed source states, the target state, **who may trigger it** (parent, AI, admin, system, provider) and the guards that must hold. An illegal move returns `409 INVALID_STATE`. Each transition writes exactly one audit event in the same database transaction, and an optimistic-lock version column stops two simultaneous approvals from both succeeding.

Approval guards: quote still valid, KYC verified, recipient allowlisted, mock sanctions check clear, sandbox per-transaction and daily caps respected, a fresh MFA proof, and the hash of the approval summary the human saw. `APPROVE` is allowed to a human parent only; the AI actor gets `AI_CANNOT_APPROVE`.

1.4 Request path and the agent loop

1. The parent compares providers, creates a draft and requests approval. The assistant can explain any number and can *propose* a draft or an approval request, but a person clicks each step.
2. Requesting approval freezes a summary (recipient, amount, rate, charge code, total cost, latest send date, quote expiry) and stores its SHA-256.
3. The parent approves with `X-MFA-Proof` and `confirm_summary_hash`. If anything changed, the hash differs and the approval is refused.
4. A background worker submits the approved payment after a short cancellation window, assigns a UETR (UUID v4) and hands it to the mock provider. The simulator later sends an HMAC-signed webhook; the API verifies it and applies `SETTLED` or `FAILED`.

2. Core algorithms

2.1 True total cost (the comparator)

Convention: every rate is **BDT per 1 unit of foreign currency**. A provider selling foreign currency quotes above mid-market; the gap is the hidden margin. For one provider, with target amount `T` (foreign), mid rate `m` and provider margin `g` percent:

```
provider_rate      = m * (1 + g/100)                          (six decimals)
fx_cost_bdt        = (provider_rate - m) * T
if charge_code == OUR:   sender_extra_bdt = (intermediary_fee + receiving_fee) * m ; shortfall =
0
else (SHA or BEN):       sender_extra_bdt = 0 ; shortfall_foreign = intermediary_fee +
receiving_fee
                         (BEN also deducts the provider's fixed fee from the beneficiary)
shortfall_bdt      = shortfall_foreign * m
total_cost_bdt     = fx_cost + fixed_fee + swift_fee + sender_extra + shortfall_bdt
cost_pct           = total_cost_bdt / (T * m) * 100
parent_pays_bdt    = T * provider_rate + fixed_fee + swift_fee + sender_extra
```

Providers are ranked by `cost_pct`, then by what the parent pays, then by provider id, so the order is deterministic. A provider that cannot offer the requested charge code is quoted with its default one, and the result carries both `requested_charge_code` and `charge_code` plus a warning.

Worked example (synthetic, GBP 10,000, mid rate 160.00):

Provider	Rate	Fixed + SWIFT	Charge	Shortfall	Total cost	Cost %	Parent pays
B	161.60	3,000 + 1,500	OUR (GBP 30 intermediary)	0	BDT 25,300	1.58%	BDT 1,625,300
C	162.00	500 + 0	SHA (GBP 25 + 10 deducted)	GBP 35	BDT 26,100	1.63%	BDT 1,620,500
A	163.20	1,500 + 1,000	OUR (fees bundled)	0	BDT 34,500	2.16%	BDT 1,634,500

B ranks first, C second with a red shortfall warning (the university would receive GBP 9,965), A third. This is blueprint test T01 and the demo uses it.

Numeric rules. Money is an integer count of minor units plus a currency code, never a float. Rates are `Decimal`. Each money part is rounded **once**, half-up, at the end. Zero, negative and unsupported amounts are rejected with `422 VALIDATION_FAILED`.

2.2 Quote lifecycle

A quote stores every input, the mid-rate source and timestamp, and an input hash. It expires after 300 seconds (configurable). Approving with an expired quote returns `409 QUOTE_EXPIRED` and sends the payment back to DRAFT; the parent requotes with the same provider.

2.3 Deadline planner

A **business day** is a day that is a working day in *both* Bangladesh (Friday and Saturday off) and the host country (own weekend and holidays), from configuration.

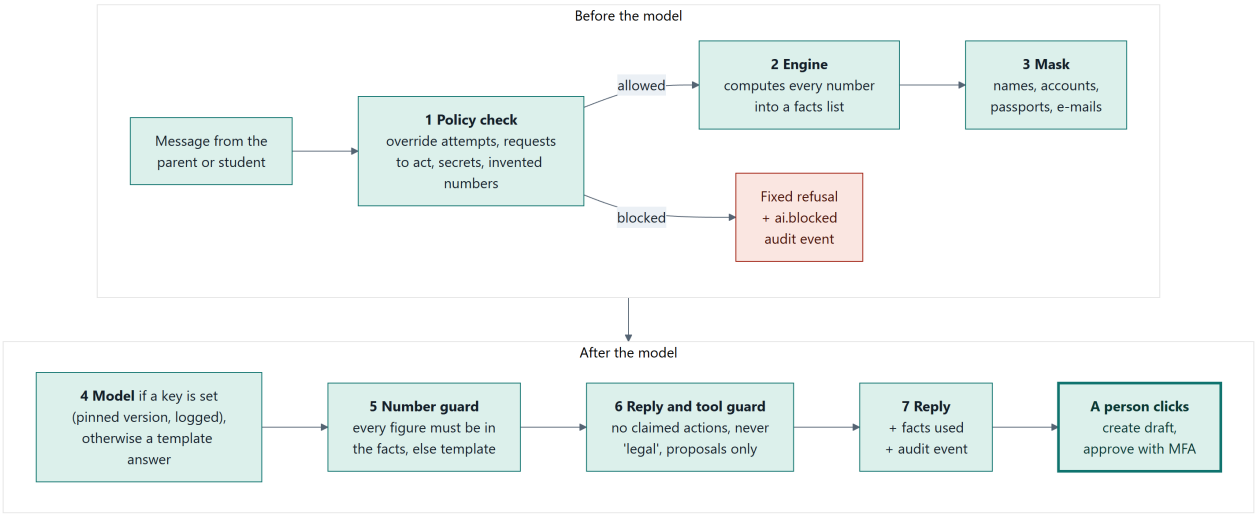
```
latest_send_date    = due_date stepped back (settlement_days + buffer_days) business days
recommended_send    = latest_send_date stepped back one more business day
earliest_send_today = today if business day and before the bank cut-off, else the next business
day
status:  ON_TRACK  if earliest <= recommended
         TIGHT     if earliest <= latest        (using the buffer)
         AT_RISK   if earliest <= latest_without_buffer
         LATE      otherwise
```

Every skipped day is returned with its reason (for example "BD weekend" or "GB holiday: ...") and shown in the UI, plus warnings for pending KYC and expired quotes. A scan limit stops a mis-configured calendar from looping forever.

2.4 Living-cost runway and synthetic rate hint

Runway reports the whole months an amount covers and the leftover (integer division of minor units), plus a one-decimal figure that is rounded **down** so it never over-promises. The rate hint compares today's synthetic rate with its recent average and says "today looks better", "about average" or "waiting may help". It is labelled synthetic and indicative, and is not a forecast.

2.5 The AI pipeline




```
 user message --> 1 policy check --> 2 facts from the engine --> 3 mask personal data
              --> 4 model (optional) --> 5 number guard --> 6 tool guard --> 7 un-mask --> 8
audit
```

1. **Policy check.** Pattern rules flag instruction overrides, action requests (approve, send, pay, cancel), limit and config changes, privilege claims, secret requests and requests to invent numbers. A

flagged message gets a fixed refusal, the model is **not called**, and an `ai.blocked` audit event is written. Document text goes through the same check.
2. **Facts.** The orchestrator builds a numbered list of facts from engine output only (`fact_17 = "Provider B cost %": "1.58%"`).
3. **Masking.** Names become `[PERSON_n]`, IBANs and long digit runs become `GB** **** 4821` style tokens, passports, e-mails and phones are removed and never restored. The mapping stays on the server and is used to un-mask the reply.
4. **Model.** Optional. A provider-agnostic client (Anthropic or an OpenAI-compatible endpoint) receives the system prompt, the masked facts and the masked message, and must answer in JSON with the ids of the facts it used. The model id is pinned by the operator and logged.
5. **Number guard.** Every number in the reply is extracted with a regular expression that ignores identifiers such as `fact_17`, normalised to a `Decimal`, and must equal a number in the allowed facts. Any other number fails the reply, which is retried once and then replaced by a deterministic **template answer** built from the same facts.
6. **Tool guard.** The model may only propose `create_draft` or `request_approval`. Reads are done by the orchestrator. Approve, submit, cancel, change limit, allowlist and edit config are not available to the AI, and each proposal must pass the same permission check as a human call.
7. **Audit.** One event records the prompt hash, output hash, model version and guard result.

With no model key the assistant runs in **template mode**: the same pipeline without step 4. The UI shows a "Template mode" badge, so the demo never depends on a network.

2.5.1 The trained reader (what the model is allowed to do)

Before step 2 above, the message is read by two readers. The rule-based reader is the baseline. A small fine-tuned language model (`bge-small-en-v1.5`, 33 million parameters, int8 ONNX, 34 MB, about 7 ms a message on CPU) predicts one of 14 intents and marks the exact words that carry an amount, a currency, a date, a period, a charge code, a purpose or a provider. Plain code (`backend/app/ai/nlu/normalize.py`) turns the marked words into typed values (integer minor units, a real date) and refuses anything it cannot read exactly. When the rules and the model read a field differently, the field is left empty and the assistant asks. The model's intent is used only above a confidence threshold (higher for anything that starts a quote, a draft or an approval request), a question about limits or legality never starts a quote, and a confident `ATTACK` can only add a block. The model has no permission of its own: the AI account still cannot approve, submit, cancel, allowlist or change a limit.

The model is trained on a GPU from generated messages plus real bank-customer questions (Banking77, CC BY 4.0) and is evaluated against the rule-based reader on held-out, natural and blind data. The design, the dataset card, the numbers (including the weak ones) and the limits are in AI_AGENT.md and AI_EVALUATION.md. With no `onnxruntime`, no model file, a failed integrity check or `NLU_MODE=off`, the assistant runs on the rules alone.

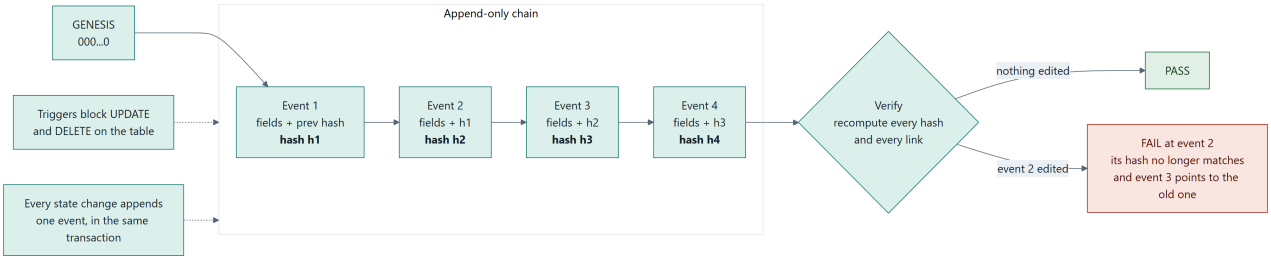
2.5.2 The dialogue policy (reinforcement learning)

When a message starts a payment plan, a second small model decides what the assistant does next: ask for the amount, the date or the period, read the understood values back and ask for a yes, or plan. It is a 30 -> 64 -> 64 -> 5 network trained with Double DQN against a simulated parent whose sentences were run through the real reader, so the noise it learns from is the system's own. The state is how each value was read (by both readers, by the model only, earlier), how sure the model was, whether the message mentioned a second amount, and the turn count. The reward is +1 for a correct plan, +0.5 for a correct

deadline or period, -2 for a wrong amount, -1 for a wrong deadline, -0.05 per question and -0.5 if the parent gives up.

The network never decides what is allowed: `backend/app/ai/dialogue/state.py` computes the legal actions in plain code (no plan with the amount missing or read two ways, no second confirmation, a send plan needs its period) and the network only ranks those. None of the five actions moves money; planning builds a quote card and a human still approves. The weights are JSON checked against a manifest hash, and with `DIALOGUE_MODE=off` or any failure the assistant plans as soon as the amount is known. Design, simulator assumptions, results and limits: AI_RL.md and AI_RL_EVALUATION.md.

2.6 Audit hash chain




```
GENESIS             = "0" * 64
canonical(e)        = JSON of e without this_event_hash, keys sorted, no whitespace, UTF-8
this_event_hash(e)  = SHA256( prev_event_hash + canonical(e) )
verify:  prev = GENESIS
         for e in events ordered by seq:
             fail at e.seq if e.prev_event_hash != prev
             fail at e.seq if SHA256(prev + canonical(e)) != e.this_event_hash
             prev = e.this_event_hash
```

Appending locks the head row, so concurrent writers keep one linear chain. `GET /audit/verify` can also compare the head with a hash published outside the database, which detects truncation. The chain is tamper-evident, not tamper-proof.

2.7 Idempotency

Every payment write needs an `Idempotency-Key` (UUID). The key, a hash of method, path and body, and the response are stored in the **same transaction** as the change, for 24 hours. The same key and body replays the stored response with `Idempotent-Replay: true` and no second effect. The same key with a different body returns `422 IDEMPOTENCY_KEY_REUSED`. A unique constraint on (user, key) makes two simultaneous requests safe.

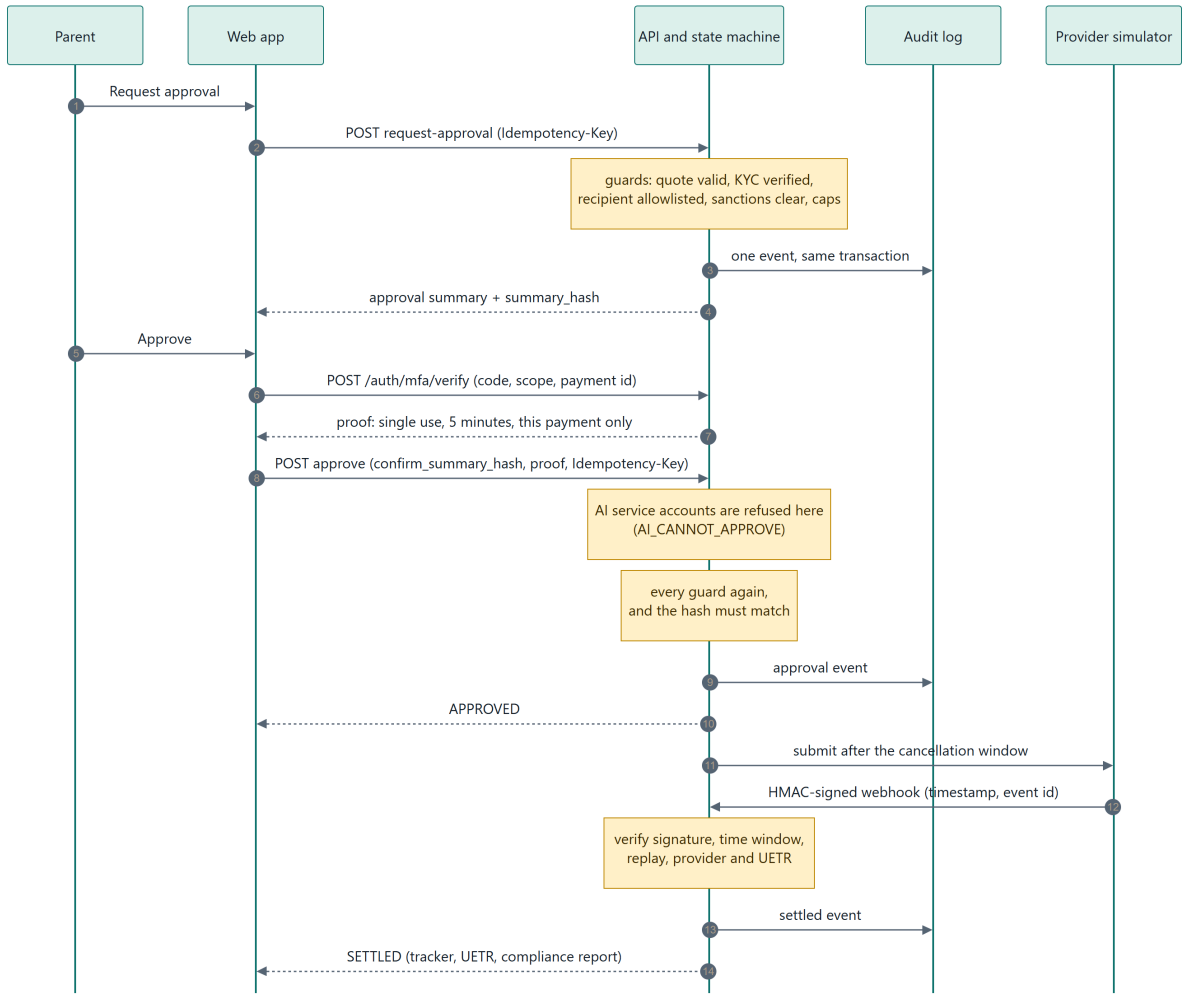
2.8 Webhook verification

```
signature = HMAC_SHA256(secret, timestamp + "." + raw_body)      header: t=<ts>,v1=<hex>
```

The receiver reads the **raw** body first, rejects a timestamp more than 300 seconds old (replay), recomputes the HMAC for the current and the previous (rotation) secret of each provider and compares in constant

time, then de-duplicates by `X-Event-Id`, and only then parses the JSON and applies the transition. Each rejection writes a `webhook.rejected` audit event and changes nothing.

2.9 MFA step-up



A fresh proof is required to approve a payment and to allowlist a recipient. It is valid 5 minutes, single use, and bound to the user, the scope (`approve_payment` or `allowlist_recipient`) and the object id, so a proof for one payment cannot approve another. TOTP (RFC 6238) works in every mode; sandbox mode also accepts a documented fixed code so judges need no authenticator app.

2.10 Smart send plan (cadence optimiser)

Fixed and SWIFT fees are paid on every transfer while the FX margin scales with the amount, so how often money is sent changes the real cost. For a total `T` over `m` months the engine considers sending monthly, every three months, every six months and all at once (duplicates collapse, so 6 months lists three options). For each option it splits `T` into equal transfers (amounts differ by at most one minor unit and sum exactly to `T`), prices every transfer with every provider using the formulas of section 2.1, and keeps the provider with the lowest total. A provider that would leave the beneficiary short is never chosen while another does not. Savings are measured against sending monthly.

Worked case (synthetic, GBP 12,000 over 12 months, charge code OUR): monthly BDT 68,400 with Provider A; every three months BDT 48,400; every six months BDT 37,800 with Provider B; all at once BDT 28,500, a

saving of BDT 39,900. The tests reproduce these numbers by hand. The engine does not predict exchange rates, so the trade-off (money sent early, rate locked early) is stated and not priced.

2.11 Agent tasks: one sentence becomes a plan

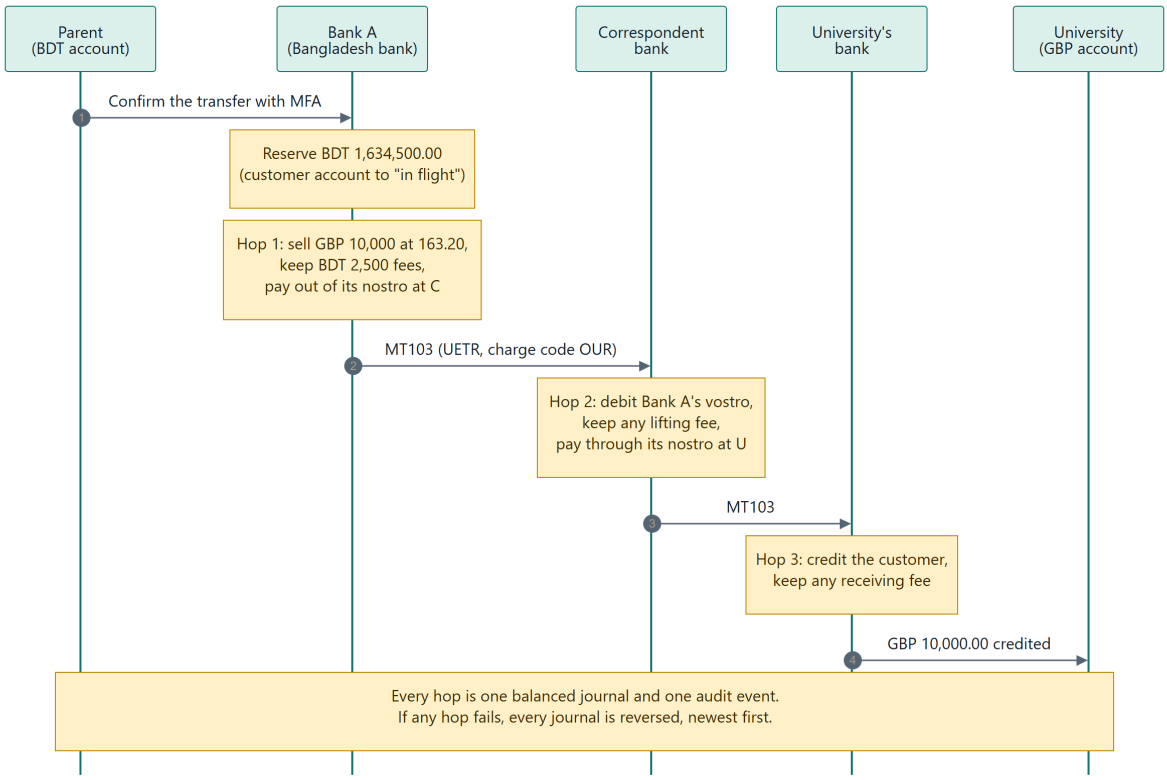
The assistant can start two tasks, chosen by a deterministic classifier (`choose_intent`) that only fires when a message really asks for one: an amount alone, a question about a limit or its legality, or a question about a quote that is already open starts nothing.

1. **Read the request.** A rule-based parser and the trained reader (section 2.5.1) each extract an amount (always through `Decimal` into minor units), currency, due date (ISO, "28 October", "5th of January", day/month/year, "in 3 weeks", "end of November", "next Friday"), purpose and charge code, and the number words, suffixes and slang the rules do not know ("ten thousand", "10k", "1.5 lakh", "quid"). Anything missing, impossible or read two ways stays empty and the assistant asks, with an example, instead of guessing. If it asked, the next message finishes the same task.
2. **Do the work with its own permissions.** For *plan a payment* it creates a quote through the same service a human uses, ranks the providers, plans the deadline, and checks KYC, saved recipients and the document checklist. For *how often should I send* it runs the cadence optimiser. Nothing else changes.
3. **Return an engine-built card.** The plan card (best provider, alternatives with shortfall warnings, deadline, seven steps marked done or to do) is built from engine output, not model text, so a model cannot alter it. Its reply text goes through the same number guard as every answer.
4. **Stop at the human gate.** The result is a `create_draft` *proposal* with the fields the human must still supply. The person creates the draft and approves with a fresh MFA code.

2.12 Compliance report

For one payment the report collects the originator and beneficiary (travel rule), ten controls each with a result (PASS, PENDING, NOT YET, BLOCKED or FAIL) and the evidence for it, the approval proof (who, when, the MFA proof reference, the summary hash), the state timeline, the payment's slice of the audit log, and the verification of the whole audit chain. It ends with the SHA-256 of the report's canonical JSON. The web screen recomputes that hash in the browser, using the same sorted-key compact JSON as the backend (a test checks the two agree byte for byte, Bengali text included), and the page prints cleanly or downloads as JSON. Reading a report writes nothing.

2.13 The bank network (interbank simulation)



`engine/interbank.py` turns a transfer request into a **plan**: a hold journal and an ordered list of steps, each with the institution that acts, the seconds before it runs, the simulated message that travels with it and the balanced journal it posts. Nothing is posted by the engine. A journal is a list of debits and credits; for every currency the debits must equal the credits, or building it raises.

National transfers run inside one country and one currency over a rail: same bank (one step), or instant, next-batch and high-value (sending bank, clearing house, receiving bank). The fee is a fixed amount plus basis points, each rail has a minimum and a maximum, and the cheapest eligible rail is chosen when none is named. *International* transfers start from a Bangladesh bank in BDT. The plan is built from the fee engine's `ProviderResult`, so the customer's debit equals `parent_pays` and the beneficiary's credit equals `beneficiary_receives` (the code raises if they ever disagree). The originating bank sells the currency, pays out of its nostro at the correspondent and keeps its fees; the correspondent forwards through its nostro at the beneficiary's bank and keeps a lifting fee; the beneficiary's bank credits the customer and keeps a receiving fee. With OUR the originating bank prefunds those two fees, with SHA they come out of the amount, with BEN the originating bank also takes its own fee from the amount.

`domain/ledger.py` applies a journal: each posting moves a balance on the account's normal side, a journal that would overdraw an account is refused before anything is written, and the journal and entry rows are append-only. A failed transfer is unwound by posting the exact opposite of every journal so far, newest first. Three checks prove integrity at any moment: the trial balance (debits equal credits per currency), the mirror accounts (each nostro equals the matching vostro, each reserve equals the clearing house's liability) and the entry sum (no stored balance differs from the sum of its entries).

`domain/bank_network.py` is the lifecycle: PENDING_CONFIRMATION, then a parent confirms with an MFA proof bound to that transfer (this reserves the funds), then ACCEPTED and IN_TRANSIT while the worker runs one step per delay, then SETTLED or RETURNED. Each state change writes one audit event in the

same transaction. All figures are integer minor units. The AI has no bank permission. The configuration (`config/bank_network.yaml`) is invented and validated when the application starts. See BANK_NETWORK.md.

2.14 Smart escrow and the audit anchor (sandbox chain)

Three Solidity contracts (`contracts/src/`) are compiled once by `contracts/build.mjs` (the `solc` WebAssembly package, `evmVersion: paris`) and the artifacts are committed, so the backend needs no compiler. `chain/evm.py` runs them on `py-evm` , a Python implementation of the EVM: one transaction is one block, a revert changes nothing, messages run without signatures or gas, and wallets are addresses derived from labels (there is no key anywhere).

The EVM is memory, so `domain/chain_service.py` keeps the history in `chain_log` : append-only and hash-chained, replayed to rebuild the chain exactly. A transaction counts only when its row commits with the request, so a request that fails halfway leaves no trace; a changed row stops the chain (`LOG_CORRUPT`). `domain/escrow.py` is the business flow: a parent plans an escrow, a human with a fresh MFA proof for that escrow locks it (mint, approve, create: three transactions, **one** audit event in the same database transaction), the worker pays each tranche on its date, and paying early or cancelling needs MFA again. The reviewer role can only freeze and unfreeze. `engine/escrow_schedule.py` splits the total in integer minor units. The AI has no escrow or chain permission. The `AuditAnchor` contract stores audit head hashes; `GET /chain/verify` reports `MISMATCH` when the database no longer agrees, even if every hash was recomputed. See SMART_ESCROW.md.

2.15 The bank review desk

Runs when approval is requested. A small neural network (`ai/risk/` , eight inputs in four factors, NumPy at run time, a SHA-256 manifest) reads the payment and rates it `LOW` , `MEDIUM` or `HIGH` from cut-offs set on a tuning period, with the groups of inputs that drove the score (found by hiding each in turn). `LOW` takes the fast lane; `MEDIUM` and `HIGH` (or a sandbox name-list match) open a `bank_reviews` row (status `OPEN` , a target time, the signals, the reading), write one audit event naming the model and the routing, and the approval guard then refuses `BANK_REVIEW_PENDING` until a reviewer clears it. The plain rules in `engine/risk_signals.py` (large amount, several payments that add up to one, a new receiver, no fee demand note) are context for the reviewer, and decide only when the model is not loaded. The reviewer (an admin user) answers `CLEAR` , `ASK_INFO` or `DECLINE` with a fresh single-use MFA proof under the scope `review_payment` ; a decline calls the existing rejection, so the state machine is unchanged. The parent's short answer is sealed with the field cipher. A clock (on time, due soon, overdue), a reminder with a cool-down and the average time to decide make a slow review visible. See `docs/BANK_REVIEW.md` and `docs/AI_RISK.md` for the permissions, the API, the results and the limits.

3. Security design

The full threat and control matrix, with the test that proves each row, is in SECURITY.md. The permission tiers and the known-risk list are in SECURITY_SELF_ASSESSMENT.md. Summary:

Concern	Design
Authentication	Argon2id passwords; ES256 access tokens (15 minutes); rotating refresh tokens with reuse detection; lockout after repeated failures; MFA step-up for approval and allowlisting
Authorisation	Role and token-scope checks in the domain layer; ownership checks return 404; the AI service account has its own scope list with no approve, cancel, reject, limit or config permission
Integrity of money	Integer minor units; summary hash bound to approval; idempotency; optimistic locking; one audit event per transition in the same transaction; double-entry journals that must balance, an append-only ledger, and a books-balance check
Provider trust	HMAC-signed webhooks with timestamp window, secret rotation and event de-duplication
AI safety	Policy check, masking, number guard, tool allow-list, human approval, template fallback
Data protection	AES-256-GCM for account numbers; masked display; log redaction filter; personal data masked before any model call; "Reset my data"
Web	Strict CSP: no inline style, and no inline script except the exported pages' own bootstrap scripts, allowed by SHA-256 hash (R42); `nosniff`; `Referrer-Policy: no-referrer`; frame-ancestors none; HSTS; exact-origin CORS without credentials; bearer tokens in memory only
Abuse	Per-IP and per-user rate limits, request size limit, schema validation that rejects unknown fields
Compliance	Rules in configuration with `source_url`, `last_checked` and a `verified` flag; yellow "Not verified" badge and "Indicative. Confirm with your AD bank." for anything unchecked; mock KYC gate; mock sanctions check; travel-rule fields in the instruction preview
Sandbox honesty	`X-Sandbox: true` on every API response and a banner on every screen; every simulated part labelled; no claim of a live bank or FX feed

4. Verification

- `backend/tests/` has one or more tests for each blueprint scenario T01 to T24 and fails if one is missing; see TEST_CASES.md.
- The dialogue policy has its own tests (`test_dialogue_*.py`, `test_rl_*.py`): the safety rules are checked exhaustively over a grid of states, a policy that always wants to plan is shown to be overruled by the rules, and the conversation (read-back, yes, no, correction, no deadline) is run end to end.
- The trained reader has its own tests (`test_nlu_*.py`, `test_ai_reader_agent.py`, `test_ml_dataset.py`) and a quality floor on held-out data (`test_nlu_quality.py`); the measured comparison with the rules is generated into AI_EVALUATION.md.
- `python -m app.scripts.demo_flow` replays the fourteen-step demo against a running API and prints OK or FAIL for each check.
- Operation logs from a real run are in `submission/evidence/`.
- `python scripts/scan_repo.py` and the CI workflow scan for secrets, real e-mails and TODOs.